

PRACTICAL - 04**Part A — Practice Programs (Hour 1)**

PROBLEM1	1. Password Strength (PasswordChecker.java + Driver.java) (a) In PasswordChecker, check four rules - length ≥ 8, contains an uppercase letter, contains a digit, contains a special character - using regex (e.g. <code>[A-Z]</code>.) or String methods. (b) Write <code>strength(String pw)</code> that counts how many rules pass and returns "Weak" (0-1), "Medium" (2-3) or "Strong" (4). (c) In <code>Driver.main</code>, test several passwords. For each, print which rules passed and the final label. Expected Output: "abc" -> Weak; "Abcd1234!" -> Strong.
CODE	<pre> public class PasswordChecker { public static boolean hasLength(String pw) { return pw.length() >= 8; } public static boolean hasUppercase(String pw) { return pw.matches(".*[A-Z].*"); } public static boolean hasDigit(String pw) { return pw.matches(".*[0-9].*"); } public static boolean hasSpecial(String pw) { return pw.matches(".*[^a-zA-Z0-9].*"); } public static String strength(String pw) { int count = 0; if (hasLength(pw)) count++; if (hasUppercase(pw)) count++; </pre>

```
        if (hasDigit(pw))
            count++;

        if (hasSpecial(pw))
            count++;

        if (count <= 1)
            return "Weak";
        else if (count <= 3)
            return "Medium";
        else
            return "Strong";
    }
}
```

DRIVER CLASS

```
public class Driver {

    public static void main(String[] args) {

        String[] passwords = {
            "abc",
            "Password",
            "Pass1234",
            "Abcd1234!",
            "hello@12",
            "WELCOME1"
        };

        for (String pw : passwords) {

            System.out.println("Password: " + pw);

            System.out.println("Length >= 8 : " + PasswordChecker.hasLength(pw));
            System.out.println("Uppercase : " + PasswordChecker.hasUppercase(pw));
            System.out.println("Digit : " + PasswordChecker.hasDigit(pw));
            System.out.println("Special Char: " + PasswordChecker.hasSpecial(pw));
            System.out.println("Strength : " + PasswordChecker.strength(pw));

            System.out.println("-----");
        }
    }
}
```

OUTPUT

```

PROBLEMS 18 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\aiml\OneDrive\Desktop\CEUE203 OOP\Practical 4> javac PasswordChecker.java
>> java Driver.java
Password: abc
Length >= 8 : false
Uppercase : false
Digit : false
Special Char: false
Strength : Weak
-----
Password: Password
Length >= 8 : true
Uppercase : true
Digit : false
Special Char: false
Strength : Medium
-----
Password: Pass1234
Length >= 8 : true
Uppercase : true
Digit : true
Special Char: false
Strength : Medium
-----
Password: Abcd1234!
Length >= 8 : true
Uppercase : true
Digit : true
Special Char: true
Strength : Strong
-----
Password: hello@12
Length >= 8 : true
Uppercase : false
Digit : true
Special Char: true
Strength : Medium
-----
Password: WELCOME1
Length >= 8 : true
Uppercase : true
Digit : true
Special Char: false
Strength : Medium
-----
PS C:\Users\aiml\OneDrive\Desktop\CEUE203 OOP\Practical 4>

```

PROBLEM2

2. Chat-log Filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder.

(e) Print "Matches: N" and the built report.

Expected Output: Keyword "hello" over 3 lines (1 malformed) -> "Matches: 1" and the matching line.

CODE

```

public class ChatFilter{

    public static String filterChats(String[] logs, String keyword) {

        StringBuilder result = new StringBuilder();
        int count = 0;

        for (String line : logs) {

            String[] parts = line.split(" ", 3);

```

```
        if (parts.length < 3) {
            continue;
        }

        String time = parts[0];
        String user = parts[1];
        String message = parts[2];

        if (message.toLowerCase().contains(keyword.toLowerCase())) {
            count++;
            result.append(time)
                .append(" ")
                .append(user)
                .append(": ")
                .append(message)
                .append("\n");
        }
    }

    return "Matches: " + count + "\n" + result.toString();
}
}
```

DRIVER CLASS

```
import java.util.Scanner;

public class Driver1 {

    public static void main(String[] args) {

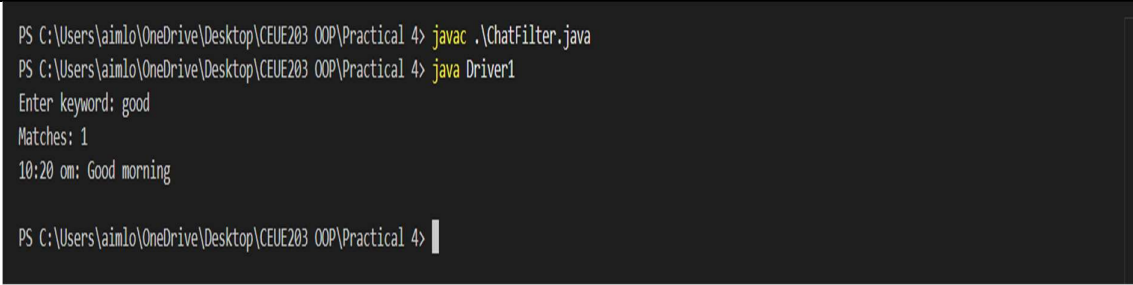
        String[] logs = {
            "10:05 om Hello there",
            "10:10 bhavy How are you?",
            "10:15",
            "10:20 manthan Good morning"
        };

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        String output = ChatFilter.filterChats(logs, keyword);

        System.out.println(output);
    }
}
```

	<pre> sc.close(); } } </pre>
OUTPUT	 <pre> PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> javac .\ChatFilter.java PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> java Driver1 Enter keyword: good Matches: 1 10:20 am: Good morning PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> </pre>
PROBLEM3	3. Template Filler (TemplateFiller.java + Driver.java) (a) In Driver.main, define a template like "Dear {name}, order {id} ships {date}." and two parallel arrays: names {"name", "id"} and values {"Riya", "A07"} (note {date} is not supplied). (b) Compile a Pattern for {(\\w+)} and get a Matcher on the template. (c) Walk the matches with a StringBuilder. For each {placeholder}, look up its value in the names array and replace it with the value, or with "[?]" if not found. (d) Print the filled template. Expected Output: The unsupplied {date} becomes [?].
CODE	<pre> import java.util.regex.*; public class TemplateFiller { public static String fillTemplate(String template, String[] names, String[] values) { Pattern p = Pattern.compile("(\\w+)"); Matcher m = p.matcher(template); StringBuilder result = new StringBuilder(); int last = 0; while (m.find()) { result.append(template.substring(last, m.start())); String key = m.group(1); String value = "[?]"; for (int i = 0; i < names.length; i++) { </pre>

```
        if (names[i].equals(key)) {  
            value = values[i];  
            break;  
        }  
    }  
  
    result.append(value);  
    last = m.end();  
}  
  
result.append(template.substring(last));  
  
return result.toString();  
}  
}
```

DRIVER CLASS

```
public class Driver2 {  
  
    public static void main(String[] args) {  
  
        String template = "Dear {name}, order {id} ships {date}.";  
  
        String[] names = {"name", "id"};  
        String[] values = {"OM", "72"};  
  
        String output = TemplateFiller.fillTemplate(template, names, values);  
  
        System.out.println(output);  
    }  
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> javac TemplateFiller.java Driver2.java  
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> java Driver2  
Dear OM, order 72 ships [?].  
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 4> █
```